

Diagrama de Clases para un sistema de fútbol (Football System).

Diagrama de Clases:

- Referee**
 - licenseNumber: String
 - certificationLevel: String
 - assignedMatches: List<Match>
 - + registerResult(hG,aG): void
 - + registerEvent(e: Event): void
 - + issueCard(p,type): void
 - + getAssignedMatches(): List
 - + isAvailable(date): boolean
 - + validateMatch(m: Match): void
 - + getProfile(): ProfileDTO
- User <<abstract>>**
 - id: Long
 - email: String
 - password: String
 - fullName: String
 - role: Role
 - profilePhoto: String
 - + login(): boolean
 - + logout(): void
 - + getUserType(): UserType
 - + getProfile(): ProfileDTO
- Player <<interface>>**
 - + validateEmail(): boolean
 - + acceptInvitation(teamId: Long): void
 - + rejectInvitation(teamId: Long): void
 - + setAvailable(available: boolean): void
 - + getUserType(): UserType
- PlayerController**
 - + register(data: RegistrationDTO): void
 - + search(id): Player
- PlayerFactory <<abstract>>**
 - # createPlayer(data: RegistrationDTO): Player
 - + registerPlayer(data: RegistrationDTO): Player
 - + validateEmail(email: String): boolean
 - + validateBasicData(data: RegistrationDTO): boolean
- AdminFactory**
 - # createPlayer(data: RegistrationDTO): Player
 - + validateEmail(email: String): boolean
- StudentFactory**
 - # createPlayer(data: RegistrationDTO): Player
 - + validateEmail(email: String): boolean
- GraduateFactory**
 - # createPlayer(data: RegistrationDTO): Player
 - + validateEmail(email: String): boolean
- TeacherFactory**
 - # createPlayer(data: RegistrationDTO): Player
 - + validateEmail(email: String): boolean
- RelativeFactory**
 - # createPlayer(data: RegistrationDTO): Player
 - + validateEmail(email: String): boolean
- AdminPlayer**
 - id: Long
 - fullName: String
 - email: String
 - password: String
 - department: String
 - position: String
 - jerseyNumber: int
 - fieldPosition: Position
 - profilePhoto: String
 - available: boolean
 - role: Role
 - teamId: Long
 - + validateEmail(): boolean
 - + acceptInvitation(teamId: Long): void
 - + rejectInvitation(teamId: Long): void
 - + setAvailable(available: boolean): void
 - + getUserType(): Player
 - + getProfile(): ProfileDTO
- StudentPlayer**
 - id: Long
 - fullName: String
 - email: String
 - password: String
 - semester: int
 - program: String
 - jerseyNumber: int
 - position: Position
 - profilePhoto: String
 - available: boolean
 - role: Role
 - teamId: Long
 - + validateEmail(): boolean
 - + acceptInvitation(teamId: Long): void
 - + rejectInvitation(teamId: Long): void
 - + setAvailable(available: boolean): void
 - + getUserType(): Player
 - + getProfile(): ProfileDTO
- GraduatePlayer**
 - id: Long
 - fullName: String
 - email: String
 - password: String
 - graduationYear: int
 - program: String
 - jerseyNumber: int
 - position: Position
 - profilePhoto: String
 - available: boolean
 - role: Role
 - teamId: Long
 - + validateEmail(): boolean
 - + acceptInvitation(teamId: Long): void
 - + rejectInvitation(teamId: Long): void
 - + setAvailable(available: boolean): void
 - + getUserType(): Player
- TeacherPlayer**
 - id: Long
 - fullName: String
 - email: String
 - password: String
 - department: String
 - position: String
 - jerseyNumber: int
 - fieldPosition: Position
 - profilePhoto: String
 - available: boolean
 - role: Role
 - teamId: Long
 - + validateEmail(): boolean
 - + acceptInvitation(teamId: Long): void
 - + rejectInvitation(teamId: Long): void
 - + setAvailable(available: boolean): void
 - + getUserType(): Player
- RelativePlayer**
 - id: Long
 - fullName: String
 - email: String
 - gmailEmail: String
 - password: String
 - relationship: String
 - relatedMember: String
 - jerseyNumber: int
 - fieldPosition: Position
 - profilePhoto: String
 - available: boolean
 - role: Role
 - teamId: Long
 - + validateEmail(): boolean
 - + acceptInvitation(teamId: Long): void
 - + rejectInvitation(teamId: Long): void
 - + setAvailable(available: boolean): void
 - + getUserType(): Player
- Team**
 - id: Long
 - name: String
 - uniformColors: String
 - shieldUri: String
 - paymentStatus: PaymentStatus
 - players: List<Player>
 - captain: Player
 - tournamentId: Long
 - points: int
 - matchesPlayed: int
 - matchesWon: int
 - matchesDrawn: int
 - matchesLost: int
 - goalsFor: int
 - goalsAgainst: int
 - + invitePlayer(playerId: Long): Invitation
 - + addPlayer(player: Player): void
 - + removePlayer(playerId: Long): void
 - + validateMinPlayers(): boolean
 - + validateMaxPlayers(): boolean
 - + validateComposition(): boolean
 - + getGoalDifference(): int
 - + getAvailablePlayers(): List<Player>
- Tournament**
 - id: Long
 - name: String
 - startDate: LocalDate
 - endDate: LocalDate
 - registrationDeadline: LocalDate
 - teamCost: Double
 - maxTeams: int
 - status: TournamentStatus
 - regulations: String
 - fields: List<String>
 - sanctions: String
 - teams: List<Team>
 - matches: List<Match>
 - + start(): void
 - + finish(): void
 - + registerTeam(team: Team): void
 - + closeRegistration(): void
 - + generateBracket(): List<Match>
 - + getStandings(): List<Team>
 - + getCalendar(): List<Match>
 - + validateDates(): boolean
 - + validateCapacity(): boolean
- Payment**
 - id: Long
 - teamId: Long
 - status: PaymentStatus
 - uploadDate: LocalDate
 - reviewDate: LocalDate
 - observations: String
 - reviewedBy: String
 - + approve(organizer: String): void
 - + reject(reason: String): void
 - + sendToReview(): void
 - + isApproved(): boolean
- Match**
 - id: Long
 - matchDate: LocalDateTime
 - field: String
 - homeTeam: Team
 - awayTeam: Team
 - homeGoals: int
 - awayGoals: int
 - phase: MatchPhase
 - status: MatchStatus
 - referee: String
 - tournamentId: Long
 - events: List<Event>
 - lineups: List<Lineup>
 - + registerResult(homeGoals: int, awayGoals: int): void
 - + registerEvent(event: Event): void
 - + getTopScorers(): List<Player>
 - + getYellowCards(): List<Player>
 - + getRedCards(): List<Player>
 - + determineWinner(): Team
 - + calculatePoints(): void
- MatchCommand <<interface>>**
 - + execute(): void
 - + undo(): void
 - + getDescription(): String
 - + getTimestamp(): LocalDateTime
- RegisterResultCommand**
 - match: Match
 - homeGoals: int
 - awayGoals: int
 - + execute(): void
 - + undo(): void
 - + getDescription(): String
- RedCardCommand**
 - match: Match
 - player: Player
 - minute: int
 - expelledPlayer: Player
 - + execute(): void
 - + undo(): void
 - + getDescription(): String
- YellowCardCommand**
 - match: Match
 - player: Player
 - minute: int
 - + execute(): void
 - + undo(): void
 - + getDescription(): String
- RegisterGoalCommand**
 - match: Match
 - scoringTeam: Team
 - previousHomeGoals: int
 - previousAwayGoals: int
 - + execute(): void
 - + undo(): void
 - + getDescription(): String
- EmailValidator <<interface>>**
 - + validate(email: String): boolean
- StudentEmailValidator**
 - domain: String
 - + validate(email): boolean
 - + getDomain(): String
- GmailValidator**
 - + validate(email): boolean
 - + getDomain(): String
- AdminValidator**
 - + validate(email): boolean
 - + getDomain(): String

Diagrama de Secuencia:

El diagrama de secuencia muestra la interacción entre los objetos durante el proceso de registro y validación de un jugador. Los participantes involucrados son: **Referee**, **User**, **PlayerController**, **PlayerFactory**, **AdminFactory**, **StudentFactory**, **GraduateFactory**, **TeacherFactory**, **RelativeFactory**, **Team**, **Tournament**, **Payment** y **Match**.

La secuencia de mensajes es la siguiente:

- Referee** llama a **registerResult(hG,aG)** en **Match**.
- Referee** llama a **registerEvent(e)** en **Match**.
- Referee** llama a **issueCard(p,type)** en **Match**.
- Referee** llama a **getAssignedMatches()** en **Match**.
- Referee** llama a **isAvailable(date)** en **Match**.
- Referee** llama a **validateMatch(m)** en **Match**.
- Referee** llama a **getProfile()** en **Match**.
- Referee** llama a **registerResult(hG,aG)** en **Match**.
- Referee** llama a **registerEvent(e)** en **Match**.
- Referee** llama a **issueCard(p,type)** en **Match**.
- Referee** llama a **getAssignedMatches()** en **Match**.
- Referee** llama a **isAvailable(date)** en **Match**.
- Referee** llama a **validateMatch(m)** en **Match**.
- Referee** llama a **getProfile()** en **Match**.
- Referee** llama a **registerResult(hG,aG)** en **Match**.
- Referee** llama a **registerEvent(e)** en **Match**.
- Referee** llama a **issueCard(p,type)** en **Match**.
- Referee** llama a **getAssignedMatches()** en **Match**.
- Referee** llama a **isAvailable(date)** en **Match**.
- Referee** llama a **validateMatch(m)** en **Match**.
- Referee** llama a **getProfile()** en **Match**.
- Referee** llama a **registerResult(hG,aG)** en **Match**.
- Referee** llama a **registerEvent(e)** en **Match**.
- Referee** llama a **issueCard(p,type)** en **Match**.
- Referee** llama a **getAssignedMatches()** en **Match**.
- Referee** llama a **isAvailable(date)** en **Match**.
- Referee** llama a **validateMatch(m)** en **Match**.
- Referee** llama a **getProfile()** en **Match**.
- Referee** llama a **registerResult(hG,aG)** en **Match**.
- Referee** llama a **registerEvent(e)** en **Match**.
- Referee** llama a **issueCard(p,type)** en **Match**.
- Referee** llama a **getAssignedMatches()** en **Match**.
- Referee** llama a **isAvailable(date)** en **Match**.
- Referee** llama a **validateMatch(m)** en **Match**.
- Referee** llama a **getProfile**